

UNIVERSIDADE ESTADUAL DE MONTES CLAROS – UNIMONTES
CENTRO DE CIÊNCIAS EXATAS E TECNOLÓGICAS – CCET
DEPARTAMENTO DE CIÊNCIAS DA COMPUTAÇÃO – DCC

EXERCICIO PROVA I

RAMON LOPES DE QUEIROZ

MONTES CLAROS – MG

ABRIL/2026

RAMON LOPES DE QUEIROZ

EXERCICIO PROVA I

Atividade avaliativa apresentada para
atendimento de requisito parcial para aprovação
na disciplina AEDS II do Curso de Graduação
em Bacharelado em Sistemas de Informação –
2º período

Professor: Dr. Heveraldo Rodrigues de Oliveira

MONTES CLAROS – MG

ABRIL/2026



Exercício da Prova Passada 1

Nome: _____ Data: 23/04/2026

1. Estruturas dinâmicas permitem armazenar vários dados na memória do computador com o benefício de dispensar a necessidade de prévio dimensionamento, este tipo de estrutura pode crescer quando precisamos armazenar novos elementos e pode diminuir quando retiramos elementos. Considere as afirmativas abaixo sobre estruturas dinâmicas:

I- A principal desvantagem de uma lista encadeada em comparação com um vetor é o tempo de acesso a um elemento em uma posição específica.

II- A alocação de memória para os nós de uma lista encadeada é feita de forma contígua, ou seja, um nó é armazenado logo após o outro na memória física.

III- Em uma Pilha (LIFO - Last-In, First-Out), a operação de remoção (pop) remove o elemento que está há mais tempo na estrutura.

IV- Em uma lista simplesmente encadeada, a inserção de um novo elemento no meio da lista exige a atualização de, no máximo, dois ponteiros.

V- Uma Fila implementada com vetor e com controle circular de índices não pode ser considerada uma estrutura dinâmica, pois seu tamanho máximo é pré-definido.

Justifique as falsas:

2. Mostre a simulação das variáveis e saída do console (vídeo), passo a passo, da execução do programa abaixo:

```
#include <stdio.h>
int Mystery(int n){
    if(n < 0) return 0;
    if(n == 0) return 1;
    printf("n = %d\n", n);
    return n + Mystery(n-2);
}
int main(){
    int X = 7, Result;
    Result = Mystery(X);
    printf("\nResultado = %d\n", Result);
    return 0;
}
```

3. Escreva uma função recursiva chamada SomaSerie() que aceite um argumento inteiro positivo n. A função deve retornar a soma da série $S = 1 + 1/2 + 1/3 + \dots + 1/n$. Use a seguinte definição recursiva:

Se $n = 1$, a soma é 1.

Se $n > 1$, a soma é $(1/n)$ somado ao resultado da função para $(n-1)$.

4. Considerando a lista encadeada cujo nó é representado pela estrutura a seguir:

```
struct lista{
    int info;
    struct lista* prox;
};
typedef struct lista Lista;

void lst_imprime (Lista* li){
    Lista* p;
    for(p = li; p != NULL; p = p->prox)
        printf("info = %d\n", p->info);
}
```

a) Implemente uma função que retorne o endereço do nó que contém a última ocorrência de um valor v na lista. Se o valor não for encontrado, a função deve retornar NULL.

```
Lista* lst_BuscaUltima(Lista* li, int v);
```

b) Implemente uma função que exclua o segundo elemento da lista. Se a lista tiver menos de dois elementos, ela não deve ser alterada. A função deve retornar o ponteiro para o início da lista.

```
Lista* lst_ExcluiSegundo(Lista* li);
```

Obs.: Para os itens (a) e (b), as funções devem percorrer a lista apenas uma vez.

5. Considere a existência dos tipos abstratos Pilha e Fila, ambos de números inteiros, cujas interfaces estão definidas nos arquivos pilha.h e fila.h da seguinte forma:

<pre>typedef struct pilha Pilha; Pilha* pilha_cria(void); void pilha_push(Pilha* p, int v); int pilha_pop(Pilha* p); int pilha_vazia(Pilha* p); void pilha_libera(Pilha* p);</pre>	<pre>typedef struct fila Fila; Fila* fila_cria(void); void fila_insere(Fila* f, int v); int fila_retira(Fila* f); int fila_vazia(Fila* f); void fila_libera(Fila* f);</pre>
---	--

Escreva uma função chamada InverteComPilha que receba uma Fila como argumento. A função deve utilizar uma Pilha auxiliar para inverter a ordem dos elementos na Fila. Ao final, a Fila original deve conter os mesmos elementos, mas na ordem inversa. A função não retorna valor.

Exemplo:

Fila de entrada: 10 -> 20 -> 30 -> 40
Fila de saída: 40 -> 30 -> 20 -> 10

```
void InverteComPilha(Fila *f);
```

6. Implemente uma função recursiva que retorne a soma de todos os valores armazenados nos nós-folhas de uma árvore binária passada como argumento. Essa função deve obedecer ao protótipo:

```
int SomaValoresFolhas(Arv *a);
```

Exercício da Prova Passada I-

I - I. V.

II - F. Os nós são alocados dinamicamente em diferentes partes da memória (não contíguas), e são conectados entre si através de ponteiros.

III - F. A LIFO remove primeiro o que entrou por último. O que remove o mais antigo é o FIFO.

IV - V.

V - V.

8 - $m = 7$

$m = 5$

$m = 3$

$m = 1$

Resultado = 16

3 - #include <stdio.h>

#include <stdlib.h>

double DomPerie (int m) {

double somaTotal;

if (m <= 1) {

somaTotal = 1.0;

} else {

somaTotal = (1.0/m) + DomPerie (m-1);

}

return somaTotal;

}

```

1 1
int main() {
    int m;
    double total;
    scanf("%d", &m);
    total = Fibonacci(m);
    printf("%.4lf\n", total);
    return 0;
}

```

```

4- struct lista {
    int info;
    struct lista* prox;
};

```

```

typedef struct lista lista;

```

```

void list_imprimir(lista* li) {
    lista* p;
    for (p = li; p != NULL; p = p->prox)
        printf("info = %d\n", p->info);
}

```

```

a- lista* list_BuscaUltima(lista* li, int no) {
    lista* p; lista* ultimo = NULL;
    for (p = li; p != NULL; p = p->prox) {
        if (p->info == no) ultimo = p;
    }
    return ultimo;
}

```



```

4. Lista * Del - Excluir Segundo (Lista * li) {
    if (li == NULL || li->prox == NULL) {
        return li;
    }
    Lista * segundo = li->prox;
    li->prox = segundo->prox;
    free(segundo);
    return li;
}

```

```

5. void Inverte Com Pilha (Fila * f) {
    Pilha * aux = pilha_cria();
    while (!fila_vazia(f)) {
        int ar = fila_retira(f);
        pilha_push(aux, ar);
    }
    while (!pilha_vazia(aux)) {
        int re = pilha_pop(aux);
        fila_insere(f, re);
    }
    pilha_libera(aux);
}

```

```

6. int ContaValoresFolhas (Arvore * a) {
    if (a == NULL) return 0;
    if (a->Esq == NULL && a->Dir == NULL)
        return a->Info;
    return ContaValoresFolhas(a->Esq) + ContaValoresFolhas(a->Dir);
}

```